4

PART 1: FOUNDATION

Modeling the business objects a system runs on

What must AI know about your business objects before it can implement anything correctly?

At Riverside Clinics, a developer picks up the Schedule Appointment use case from Chapter 3 and hands it to an AI assistant with one instruction: implement this. The use case names the actor, the preconditions, the seven-step booking flow, the ninety-day window clinic operations agreed to. It reads like exactly the specification-shaped request Chapter 1 asked for. The code that comes back compiles, passes its own tests, and looks finished. Nobody catches what it got wrong until a patient shows up at the wrong site: the code assumes every doctor works exactly one clinic, because nothing in the use case said otherwise, and half of Riverside's doctors rotate between two.

CHAPTER PROMISE

By the end of this chapter, you can build a domain model, entities, value objects, enumerations, and the relationships between them, precise enough that an AI assistant stops inventing your business objects and starts implementing the ones you already have.

Reading time: 10 minutes

What AI invents when nobody names the objects

A use case tells AI what happens: who does what, in what order, under what conditions. It does not tell AI what those actors and things are. The Schedule Appointment use case says a patient books an open slot with an eligible doctor. It never says whether a doctor can work more than one clinic, what values an appointment's status can take, or whether a patient record and a medical record are the same object or two objects that travel together. Somebody has to answer these questions before the code is correct. If nobody does, AI answers them anyway.

Its answers are guesses shaped like decisions. Riverside's AI assistant guessed a one-to-one relationship between doctor and clinic, because the use case never mentioned two, and one is the simpler assumption to build against. It guessed appointment statuses of "Booked" and "Done," because those are ordinary values in appointment systems generally, not because anyone at Riverside uses those words. The front desk staff who use Riverside's existing systems call them "Scheduled" and "Completed." Neither guess broke a test. Both guesses were wrong.

The cost shows up downstream, well after the pull request closes. A report built against "Booked" returns zero appointments, because nothing in the database matches that value. A doctor who splits her week between two clinics gets scheduled as if she only worked one, and a patient books a slot the doctor isn't actually staffing that day. The tests were never going to catch this. It is a business fact nobody told the AI, arriving in the finished system as if the AI had decided it, because it had.

What a domain model actually describes

A **domain model** describes the business objects a system deals with, independent of how any database happens to store them: what things exist, what facts are true about each one, and how they relate to each other. It stays the same regardless of which entity-relationship diagram a particular database uses, or which class hierarchy a particular language happens to prefer. Riverside's domain model says the same thing whether the system behind it runs on Postgres or MongoDB, Java or Python, because it describes the clinic's business rather than any one technology's way of representing it.

The model is built from three different kinds of business fact, and mixing them up is a common way domain models go wrong. An **entity** is something the business tracks as itself over time, something with its own identity that persists even as its attributes change: a Patient stays the same patient after a phone number changes, a Doctor stays the same doctor after moving offices. A **value object** is a fact fully described by its own value, with no identity of its own to track, and replaced rather than edited when it changes: two appointments scheduled for the same fifteen-minute window on the same day describe the identical time slot, the same fact recorded twice rather than two different facts that happen to look alike. An **enumeration** is an attribute limited to a fixed, named set of valid values: an appointment's status is Scheduled, Completed, Cancelled, or NoShow, never a fifth value someone typed by hand.

None of this is new. The distinction between entities and value objects, and the discipline of naming a domain model before writing code, comes from established software design practice, most visibly the domain-driven design tradition Eric Evans described. Riverside doesn't need to reinvent it. It needs to apply it once, clearly, to patients, doctors, and appointments.

> **A domain model names what your business already believes to be true. It doesn't invent anything, and neither should the AI reading it.**

Building the model in four passes

1. List every noun your use cases already depend on: patient, doctor, appointment, clinic, time slot, without deciding yet what kind of fact each one is.
2. Classify what you listed. For each noun, ask whether it needs an identity you'll refer back to later (an entity), whether it's fully described by its current value with no identity of its own (a value object), or whether it's really just an attribute limited to a fixed set of options (an enumeration).
3. For each entity and value object, name its attributes concretely, not "details" but "phone_number," and name its relationships to other entities, including the cardinality on each side: exactly one, one or more, or many.
4. Check the model against your use cases in both directions. Every noun a use case depends on should appear in the model. Every entity in the model should be used by at least one use case; if one isn't, remove it.

The Riverside Clinics domain model, named once

Run patients, doctors, appointments, clinics, and time slots through the four passes, and here is what the model looks like for the slice of Riverside's business the Schedule Appointment use case touches. This is an excerpt. Riverside's real model also covers billing and insurance. It's complete enough, though, that an AI assistant implementing the booking flow no longer has to guess any of it.

Riverside Clinics domain model (excerpt)

- **Patient**: a person who receives care. Attributes: medical_record_number (unique), date_of_birth. Relationship: has many Appointments.
- **Doctor**: a clinician who provides care. Attributes: specialty (enumeration: Cardiology, Dermatology, Orthopedics, Family Medicine, Psychiatry), years_licensed. Relationship: works at one or more Clinics (many-to-many).
- **Appointment**: a scheduled visit. Attributes: scheduled_slot (TimeSlot value object), status (enumeration: Scheduled, Completed, Cancelled, NoShow). Relationship: involves exactly one Patient and exactly one Doctor, at exactly one Clinic.
- **Clinic**: one physical Riverside location. Attributes: name, business_hours. Relationship: employs many Doctors.
- **TimeSlot** (value object): a fifteen-minute window with no identity of its own. Attributes: date, start_time, end_time. Meaningful only as the scheduled_slot of an Appointment.

Compare this to what the AI assistant guessed at the start of this chapter. The relationship between Doctor and Clinic is many-to-many here, because the model says so in writing instead of leaving the AI to assume the simpler case. The status values are Scheduled, Completed, Cancelled, and NoShow, in the front desk's own words, the words the people who actually run the clinics use every day.

Where a domain model still goes wrong

▲ **WARNING**

A domain model can quietly turn into a database diagram. Riverside's real database splits a patient's address into its own table, to avoid storing the same address twice for a household sharing one home. The domain model still says, correctly, that a patient has one address. How the database avoids duplicating it is a storage decision, made later, by different people, for different reasons.

Modeling the database instead of the business	The model inherits the schema's normalization: foreign keys as relationships, join tables as entities of their own. An AI implementing the next feature then copies structure that was never a business decision in the first place.
Freezing the model after the first draft	Riverside adds telehealth visits eighteen months later, and Scheduled, Completed, Cancelled, and NoShow no longer cover what actually happens. A video visit that never connects sits between NoShow and Cancelled, and nobody updated the enumeration to say which one it is.

Field guide: telling an entity from a value object

- [ ] Does it need an identity you'd refer back to later, even after every one of its attributes changes? If so, it's an entity.
- [ ] Do two instances with identical attributes mean the same thing, or two different things that happen to look alike? The same thing means a value object; two different things mean an entity.
- [ ] Does it ever exist on its own, apart from the entity it describes? If it only ever appears as part of something else, it's a value object.
- [ ] When one of its parts changes, do you edit it in place or record a new one? A value object gets replaced whole; an entity gets edited and stays the same entity.
- [ ] Is it really just an attribute limited to a small, fixed, named set of options? Then it's an enumeration, neither an entity nor a value object.

Run this checklist noun by noun while you're building the model, before the code comes back mismatched.

What to remember

- Without a domain model, AI has to guess your entities, attributes, and relationships, and most of what it guesses is wrong for your business specifically, however ordinary each guess looks on its own.
- A domain model names business meaning; it is a different document from a database schema, and the two are allowed to disagree.
- Entities carry identity across changes, value objects are fully described by their own value, and enumerations limit an attribute to a fixed set of options. The model earns its keep once every business object is sorted correctly among the three.
- The model changes as the business changes. A Riverside that starts offering telehealth visits needs its Appointment status enumeration revisited then, on the same timeline as the change itself.

QUESTIONS FOR YOUR TEAM

- Which of your system's core objects exist only in a database migration file, invisible to any document a new hire could read in an afternoon?
- The last time your business added a new kind of record, telehealth, a new appointment status, a new product line, did anyone update the domain model, or only the code?

ONE ACTION TO TAKE NEXT

Pick the one entity in your system most likely to be modeled wrong right now, the one everyone assumes they already understand. Run it through this chapter's checklist before you trust an AI assistant with it again.

TRANSITION. Chapter 3 gave the system a use case. This chapter gave it a domain model. The next chapter puts a vision, requirements, use cases, and a domain model into one document, and follows the Riverside Clinics booking feature through all four layers at once.

Modeling the business objects a system runs on ends here.